

# How to Learn the Mathematics of Machine Learning

## Contents

|                                                       |   |
|-------------------------------------------------------|---|
| 1. Overview . . . . .                                 | 2 |
| 2. Habit 1: change the attitude . . . . .             | 3 |
| 2.1 Reason 1: we always know why . . . . .            | 3 |
| 2.2 Reason 2: only a small subset is needed . . . . . | 3 |
| 3. Habit 2: decode the notation . . . . .             | 4 |
| 4. Habit 3: use code to see the maths . . . . .       | 5 |
| 5. Habit 4: learn visually first . . . . .            | 6 |
| 6. Habit 5: learn in context . . . . .                | 6 |
| 7. Summary . . . . .                                  | 8 |
| 8. Key terms . . . . .                                | 8 |

**Where this fits:** Pipeline map step 0 (Foundations). See the [Course map](#).

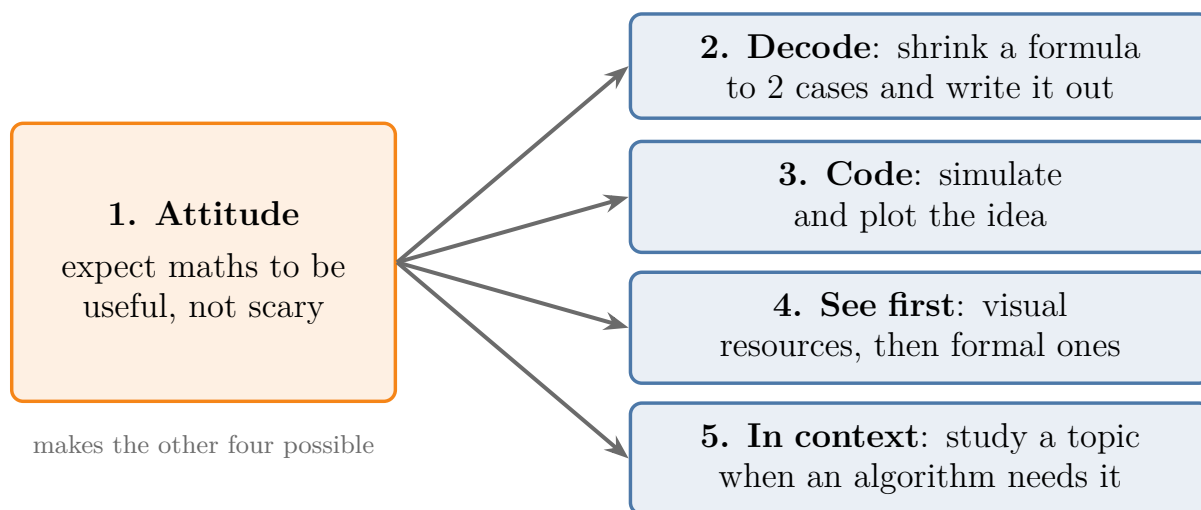


- **Builds on:** Role of mathematics in ML ([Note 440](#)).

## 1. Overview

**Key point:** The fear of mathematics fades with five habits: expect maths to be useful, decode notation on small cases, check ideas with code, learn visually first, and study each topic in the context where ML needs it.

Many learners want to work in ML but stop at the first equation. The maths ML needs is smaller and easier than it looks, and the right way of studying it matters more than talent.



**Figure 1:** Five habits for learning the maths of ML; the first one makes the other four possible

Figure 1 shows the five habits. The first one, the attitude, comes first because the others only work once we are willing to spend time with the maths.

Two earlier Notes already cover parts of this topic. The [role of mathematics in ML Note](#) explains the job of each branch of maths in ML. The [statistics roadmap Note](#) and the [linear algebra roadmap Note](#) list the topics of those two branches. This Note adds how to study them, and the calculus part of the map.

## 2. Habit 1: change the attitude

**Key point:** Instead of quitting at the first equation, we expect it to explain the algorithm; two facts make this realistic: in ML we always know why a piece of maths is there, and only a small, easy part of maths is needed.

The usual pattern is: open an algorithm, meet an equation, decide it is too hard, and stop. The habit to build is the opposite reaction: an equation is where the algorithm is actually explained.

A practical trick is to picture a future version of ourselves who reads equations comfortably. Aiming at that picture makes it easier to stay with a formula for ten minutes instead of five, and to work through it with pen and paper. After a few weeks of this, most learners find the maths of ML far less hard than its reputation.

### 2.1 Reason 1: we always know why

**Key point:** School maths feels dry because we rarely know what it is for; in ML every piece of maths has a visible purpose.

At school we solve hundreds of derivatives and integrals for the exam, without knowing where they are used. Maths without a purpose is boring, and boring maths feels hard.

In ML the purpose is always visible:

- **Derivatives** are there to minimise a loss function, which is how a spam filter learns to separate spam from normal mail (see the [gradient descent Note](#)).
- **Matrices** are there to transform data; a photo filter, for example, is a manipulation of the matrix of pixel values (see the [linear transformations and matrices Note](#)).

Because we know why each formula is there, it stays interesting, and interesting maths is easier to learn.

### 2.2 Reason 2: only a small subset is needed

**Key point:** ML uses four topics: statistics, probability, linear algebra (mostly matrices) and a small part of calculus (differentiation, for optimisation).

We do not need all of mathematics. ML uses four topics, and only part of each:

| Topic          | How much                                             | Where the map of it is                      |
|----------------|------------------------------------------------------|---------------------------------------------|
| Statistics     | know it well                                         | <a href="#">statistics roadmap Note</a>     |
| Probability    | a fair idea                                          | <a href="#">events Note</a> onward          |
| Linear algebra | know it well, mostly matrices                        | <a href="#">linear algebra roadmap Note</a> |
| Calculus       | a small part: differential calculus for optimisation | Section 6 of this Note                      |

Compared with a field such as theoretical physics, this maths is both smaller in scope and less abstract. Less to learn, easier to learn, and always with a reason: together these make the maths enjoyable, and enjoyment removes the fear.

### 3. Habit 2: decode the notation

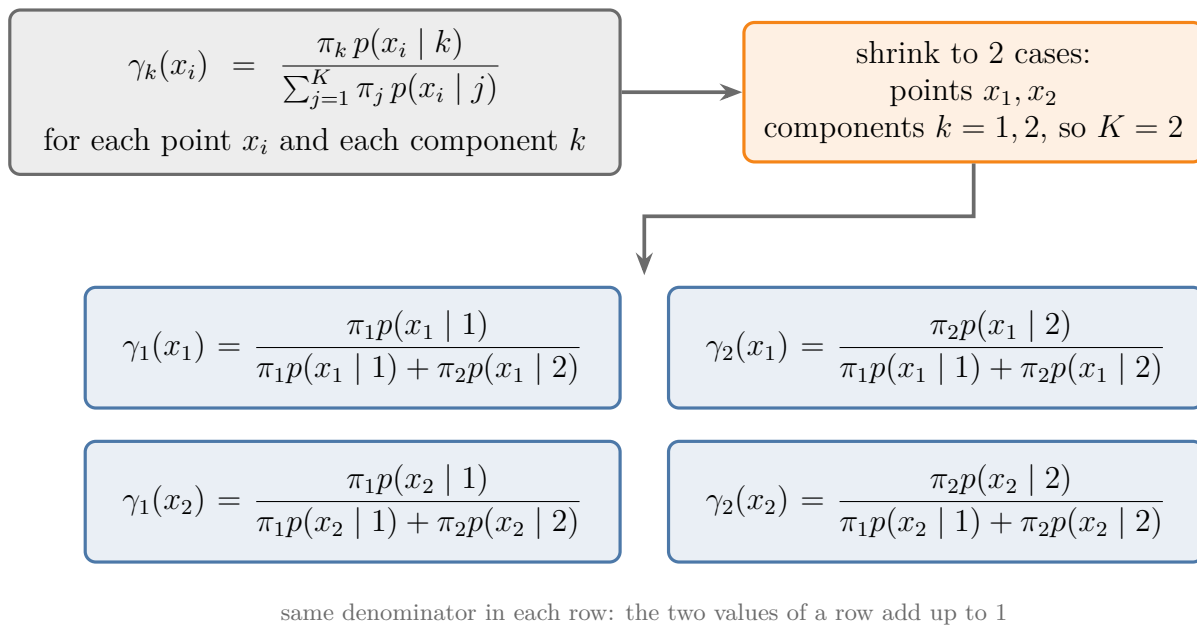
**Key point:** When a formula full of indices looks frightening, we shrink it to two or three cases and write every case out by hand.

The second source of fear is notation: Greek letters, indices and sums packed into one line. The notation will not go away, because it is the language of maths. What we can learn is to unpack it.

The method:

1. Read the sentence around the formula (“for each point  $x_i$  and each component  $k$ ”).
2. Replace every index range by a tiny one: two points, two components.
3. Write out every case the formula stands for, with the indices filled in.

Figure 2 applies it to a formula from an algorithm we have not met yet, Gaussian mixture models. We do not need to know the algorithm; the point is the unpacking.



**Figure 2:** Decoding a formula: the general form (top left) shrunk to two points and two components, then written out case by case

- “For each point and each component” means one value  $\gamma_k(x_i)$  per pair: four values for two points and two components.
- The sum  $\sum_{j=1}^K$  in the denominator becomes two terms, because  $K = 2$ .

- Written out, each row has the same denominator, so the two values of a row add up to 1: the formula splits each point between the components.

Worked with numbers: take  $\pi_1 = 0.6$ ,  $\pi_2 = 0.4$ ,  $p(x_1 | 1) = 0.5$  and  $p(x_1 | 2) = 0.2$ . Then

$$\gamma_1(x_1) = \frac{0.6 \times 0.5}{0.6 \times 0.5 + 0.4 \times 0.2} = \frac{0.30}{0.38} = 0.79, \quad \gamma_2(x_1) = \frac{0.08}{0.38} = 0.21$$

and  $0.79 + 0.21 = 1$ , as the written-out form predicted. A line of compact notation became four plain fractions.

**Python:** a loop is the same unpacking, done by the computer. Each line below is one box of Figure 2.

```
import numpy as np

pi = np.array([0.6, 0.4])          # weight of each component
p = np.array([0.5, 0.2])          # p(x1 | k) for k = 1, 2
gamma = pi * p / (pi * p).sum()    # both gamma_k(x1) at once
print(gamma.round(2))              # [0.79 0.21]
```

#### 4. Habit 3: use code to see the maths

**Key point:** A small simulation or interactive plot turns an abstract idea into something we can change and watch.

Once we can program a little, every concept can be checked by building a small tool around it: change an input, watch the output. Ideas that are hard to picture become concrete. Examples where this helps most:

- **Confidence intervals:** simulate many samples and count how often the interval contains the true mean (see the [interpreting confidence intervals Note](#)).
- **Matrices as transformations:** type in a matrix and watch the grid move (see the [linear transformations and matrices Note](#)).
- **The t distribution:** plot it next to the normal curve and raise the degrees of freedom until the two match (see the [t procedure Note](#)).
- **Dividing by  $n - 1$  in the sample variance:** simulate many samples and compare the averages of both versions (see the [measures of dispersion Note](#)).

We do not need to be strong programmers for this. Many such tools are shared on GitHub, and an AI assistant can write one from a prompt such as “write a Streamlit app that plots the normal and t distributions side by side, with a slider for the degrees of freedom”. We run it, then fix and extend it step by step.

## 5. Habit 4: learn visually first

**Key point:** First get the picture from a visual resource, then read the formal treatment in lectures or books.

University lectures and textbooks are accurate, but most of them write equations and solve them, with few pictures. Starting with a visual explanation (diagrams, animation) and only then turning to the formal version makes both easier.

Three visual resources cover most of what ML needs:

- **Khan Academy.** The school mathematics chapters ML uses: statistics, probability, limits and derivatives, and vector algebra.
- **3Blue1Brown.** Two series: *Essence of Linear Algebra* (worth watching several times before deep learning) and *Essence of Calculus*.
- **StatQuest (Josh Starmer).** A statistics series of about 60 lessons, animation first and definitions after.

After the picture, the formal sources: university lectures (for example NPTEL or Stanford) and books. The roadmap Notes list books for [statistics](#) and [linear algebra](#).

## 6. Habit 5: learn in context

**Key point:** We do not study a whole subject before starting ML; we study each topic knowing where ML uses it, and leave the rest until an algorithm needs it.

Even within four topics, a common mistake is to try to learn everything in each: all of statistics, then all of linear algebra. That takes months and loses the “why” of Habit 1.

A roadmap fixes this. For each topic it lists the parts ML uses and where each part is used, so we can find the best resource for exactly that part. The [statistics roadmap Note](#) and the [linear algebra roadmap Note](#) are such roadmaps. The calculus part is short.

**Extra:** The calculus and optimisation part of the roadmap, in the order this project teaches it:

| Topic                                      | Where ML uses it                               | Note                     |
|--------------------------------------------|------------------------------------------------|--------------------------|
| Derivatives of one variable                | the slope of a loss                            | <a href="#">Note 600</a> |
| Partial derivatives, gradients, chain rule | gradient descent, backpropagation              | <a href="#">Note 601</a> |
| Jacobian and matrix gradients              | the gradient of a least-squares loss           | <a href="#">Note 602</a> |
| Hessian and Taylor series                  | curvature, Newton steps, XGBoost               | <a href="#">Note 603</a> |
| Gradient descent                           | training almost every model                    | <a href="#">Note 57</a>  |
| Convex and non-convex cost functions       | whether gradient descent finds the best answer | <a href="#">Note 590</a> |

|                                                   |                                                |                          |
|---------------------------------------------------|------------------------------------------------|--------------------------|
| Constrained optimisation,<br>Lagrange multipliers | SVM, the constraint view<br>of Ridge and Lasso | <a href="#">Note 620</a> |
| Convex sets, functions and<br>problems            | knowing when a minimum<br>is the global one    | <a href="#">Note 621</a> |
| Linear and quadratic<br>programming               | SVM training, resource<br>planning             | <a href="#">Note 622</a> |

Integration is rarely needed by hand: it appears mainly as the area under a probability density (see the [PDF and continuous CDF Note](#)).

## 7. Summary

| Habit         | What we do                                            | Why it works                                      |
|---------------|-------------------------------------------------------|---------------------------------------------------|
| 1. Attitude   | treat an equation as the explanation, not an obstacle | ML maths is small, easy, and always has a purpose |
| 2. Decode     | shrink indices to 2 cases and write each case out     | compact notation becomes plain arithmetic         |
| 3. Code       | simulate and plot the idea                            | we can change inputs and watch                    |
| 4. See first  | visual resources, then formal ones                    | the picture makes the formulas readable           |
| 5. In context | study a topic when an algorithm needs it              | saves months and keeps the purpose visible        |

- ML needs four topics: statistics, probability, linear algebra (mostly matrices) and differential calculus for optimisation.
- The attitude comes first; without it the other four habits do not get used.

## 8. Key terms

| Term                | Meaning                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| Decoding notation   | Shrinking a formula's indices to two or three cases and writing every case out by hand                    |
| Contextual learning | Studying a maths topic together with the ML algorithm that uses it, instead of the whole subject up front |